A PROJECT REPORT ON

“CHAINSHIELD TRANSPORTATION SYSTEM”

Resilient Logistics & Dynamic Supply Chain Optimization

Submitted in Fulfillment of the Requirement
for Completion of Semester – 3 in
Master of Computer Application (MCA)
Academic Year 2026-27

To
LJ INSTITUTE OF COMPUTER APPLICATION
Affiliated to LJK University

Guided By:

Prof. Dhaval Shah

Prepared By: (Group No. 59)

Bhavik A. Visani (D51 - 25004401110170)
Diya Shah (D36 - 25003401110017)
Shruti Somani (D42 - 25004401110259)

DECLARATION

We, the undersigned students of Group No. 59, Master of Computer Application (MCA) Semester 3 at LJ Institute of Computer Application (affiliated to LJK University), hereby declare that the project entitled “ChainShield Transportation System” is our own original work, carried out under the guidance of Prof. Dhaval Shah, and has not been submitted, in whole or in part, for the award of any other degree, diploma, or certificate to this or any other institute or university.

We further declare that all the information, technical concepts, and design decisions presented in this report have been gathered, analysed, and implemented by us to the best of our understanding, and that any reference material used has been duly acknowledged in the Bibliography.

Date: _____ Place: Ahmedabad

Bhavik A. Visani (D51)

Diya Shah (D36)

Shruti Somani (D42)

CERTIFICATE

This is to certify that the project entitled “ChainShield Transportation System”, submitted for MCA Semester 3, has been carried out by Bhavik A. Visani (D51), Diya Shah (D36), and Shruti Somani (D42) of Group No. 59, under my guidance and supervision.

The work presented in this report is a bona fide record of the project carried out by the above-named students and, to the best of my knowledge, has not been submitted elsewhere for the award of any degree or diploma.

I wish them success in their future endeavours.

_____ _____

Prof. Dhaval Shah

Project Guide

Head of Department / Director

LJ Institute of Computer Application

PREFACE

Global trade today moves through supply chains of extraordinary scale and complexity — millions of shipments crossing ports, highways, railways, and air corridors every single day. The same forces that make this scale possible also make it fragile: a single storm, a congested port, or a mishandled hazardous cargo shipment can ripple into delays that cost businesses time, money, and customer trust. Historically, these disruptions have been discovered only after the fact, when a delivery has already slipped past its promised window.

This project, ChainShield Transportation System, was conceived to address exactly that gap. Rather than treating disruption management as a reactive, after-the-event exercise, ChainShield continuously evaluates every active shipment against a weighted risk model, flags the ones heading toward trouble, and proposes concrete rerouting alternatives before the delay materialises.

The idea originated from our participation in the Smart Supply Chains track of a solution-building initiative, where we built an initial prototype of ChainShield using FastAPI, React, and Firebase. For our MCA Semester 3 project, we chose to rebuild the same problem statement from scratch on a leaner, more traditional academic stack — Python (Flask), SQLite, and server-rendered Jinja2/Bootstrap templates, containerised with Docker — so that the system is simpler to explain, defend, and run consistently on any machine.

This report documents the problem definition, requirement analysis, system design, development approach, and testing strategy for the rebuilt ChainShield Transportation System, prepared as part of the curriculum prescribed for MCA Semester 3 at LJ Institute of Computer Application.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported us through the course of this project.

First and foremost, we thank LJK University and LJ Institute of Computer Application for providing us the opportunity and the academic environment to undertake this project as part of our MCA Semester 3 curriculum.

We are deeply grateful to our project guide, Prof. Dhaval Shah, for his valuable guidance, constructive feedback, and constant encouragement throughout the planning, design, and documentation of this project. His inputs have been instrumental in shaping the direction of ChainShield Transportation System.

We also extend our thanks to the Director and faculty of LJ Institute of Computer Application for their support, and to our classmates and peers who offered helpful suggestions during discussions on this project.

Finally, we thank our families for their patience and encouragement throughout this endeavour.

Thank you,

Bhavik A. Visani, Diya Shah, Shruti Somani

Group No. 59, MCA Semester 3

INDEX

Sr.	Particulars	Page No
1	**Introduction** 1) Problem Definition 2) Objective of the New System 3) Core Components 4) Project Profile 5) Need of the New System 6) Scope of the New System 7) Advantages and Limitations of the Proposed System 8) Proposed Time Line Chart 9) Targeted User	7
2	**Requirement Determination and Analysis** 1) Requirement Determination 2) Requirement Specification	11
3	**System Design** 1) Use Case Diagram 2) Data Dictionary 3) User Interface Design 4) Report Design	13
4	**Development** 1) Coding Standards	18
5	**Testing** 1) Test Cases	19
6	**Proposed Enhancement**	21
7	**Conclusion**	22
8	**Bibliography**	23

1. INTRODUCTION

1.1 Problem Definition

Modern global supply chains manage millions of concurrent shipments across highly complex and inherently volatile transportation networks. Critical transit disruptions — ranging from sudden weather events to hidden operational bottlenecks such as port congestion, customs delays, or vehicle breakdowns — are chronically identified only after delivery timelines are already compromised.

In practice, logistics managers rely on manual tracking and reactive escalation: a disruption becomes visible only once a customer or downstream partner reports a missed delivery. By that point, the shipment is already late, alternate routing options have narrowed, and the cost of recovery — expedited freight, penalty clauses, damaged customer trust — has multiplied. There is no continuously running system that scores every active shipment's risk in real time and proposes a corrective route before the delay actually happens.

1.2 Objective of the New System

- Continuously analyse multifaceted transit data (weather, route congestion, cargo sensitivity, historic delay patterns) for every active shipment.
- Preemptively detect and flag potential disruptions before delivery timelines are compromised, using a composite, weighted risk-scoring model.
- Automatically generate optimised route-adjustment recommendations (reroute vs. hold) the moment a shipment crosses a risk threshold.
- Notify the right stakeholder — manager, then the assigned field operator — the instant a decision is made, closing the loop between detection and execution.
- Provide role-based dashboards so admins, managers, and field operators (captains, pilots, drivers, loco pilots) each see only what is relevant to their responsibilities.
- Keep the system self-contained, inexpensive to run, and simple to demonstrate, using an open-source Python/Flask stack packaged with Docker.

1.3 Core Components

- Risk Engine — scores every active shipment on a recurring schedule using a 4-factor weighted model (weather severity, route congestion, cargo sensitivity, historic delay probability).
- Disruption Manager — converts a high-risk score into a formal disruption record and generates ranked route-solution alternatives.
- Route Solver — computes waypoints and alternate-route options with estimated extra time, extra cost, and residual risk for each option.
- Notification & Chat Module — pushes in-app alerts to managers and field operators, and supports direct messaging between users.
- Admin Panel — manages users, logistics hubs, cargo types, and maintains an activity audit log.

- AI Assistant — a conversational assistant (Gemini API) that answers natural-language questions about the state of the supply chain.
- Live Map — a Leaflet.js visualisation of shipment positions, routes, and active disruption zones.

1.4 Project Profile

Project Title	ChainShield Transportation System
Domain	Smart Supply Chains — Resilient Logistics & Dynamic Supply Chain Optimization
Project Description	A system that continuously scores active shipments for disruption risk and instantly proposes optimised route adjustments before delays cascade, with role-based dashboards for admins, managers, and field operators.
Front End	HTML5, CSS3, Bootstrap 5, Jinja2 templating, JavaScript, Leaflet.js, Chart.js
Back End	Python (Flask), Flask-SQLAlchemy, SQLite
Other Tools	Flask-Login, Flask-SocketIO, APScheduler, Flask-WTF, Google Gemini API, OpenWeatherMap API, Docker
Guide	Prof. Dhaval Shah
Submitted To	LJ Institute of Computer Application (LJK University)

1.5 Need of the New System

- Manual, reactive monitoring means disruptions surface only after a customer complaint or missed SLA — by then recovery options are limited and costly.
- Existing tracking dashboards typically show where a shipment IS, not what is about to go wrong with it — there is a visibility gap between tracking and prediction.
- Weather, congestion, cargo-type, and historical-delay signals sit in silos; no single continuously-running process fuses them into one actionable risk score per shipment.
- Rerouting decisions are usually ad hoc, based on a manager's personal experience, rather than a systematic comparison of ranked options with quantified cost, time, and risk trade-offs.
- Field operators (drivers, captains, pilots) often learn about a rerouting decision late — over phone or radio — introducing further delay and communication error.

1.6 Scope of the New System

Included in this project:

- Interval-based risk scoring for every active shipment (near-real-time).
- Automatic disruption creation once a shipment crosses a risk threshold, with ranked reroute/hold solutions.
- Manager dashboard for reviewing and resolving disruptions.
- Field operator dashboard for status updates and receiving resolution notifications.

- Admin panel for user, hub, and cargo-type management, plus an audit log.
- A basic AI assistant for natural-language questions about shipment data.
- Live map visualisation of shipments and disruption zones.
- A demo/simulation mode for showing the system without needing live production data.

Outside the scope of this academic build:

- Integration with live carrier/GPS hardware feeds (simulated positions are used instead).
- Payment, billing, or contractual/SLA enforcement modules.
- Multi-tenant support for multiple independent companies.

1.7 Advantages and Limitations of the Proposed System

Advantages:

- Shifts the organisation from reactive to preemptive disruption handling.
- Quantifies risk with a transparent, weighted scoring formula instead of gut feeling.
- Cuts the time between “risk detected” and “operator notified” to seconds.
- Role-based views reduce information overload for each type of user.
- Runs on a lightweight Flask + SQLite stack, containerised with Docker, so it is cheap to host and easy to demonstrate on any machine.

Limitations:

- Near-real-time, not instantaneous — performance depends on the scheduler interval (60 seconds by default).
- Weather and route-congestion signals rely on external APIs, or a simulated fallback, rather than certified logistics-grade data feeds.
- SQLite suits a single-node academic prototype but would need to be replaced by PostgreSQL/MySQL for real multi-user production load.
- The AI assistant's answers are only as reliable as the underlying Gemini model and the data exposed to it — it is a convenience feature, not a system of record.

1.8 Proposed Time Line Chart

The project is planned across the semester using the phased schedule below. As development has not yet begun at the time of writing this report, these are proposed (not actual) dates, to be tracked and updated as work progresses.

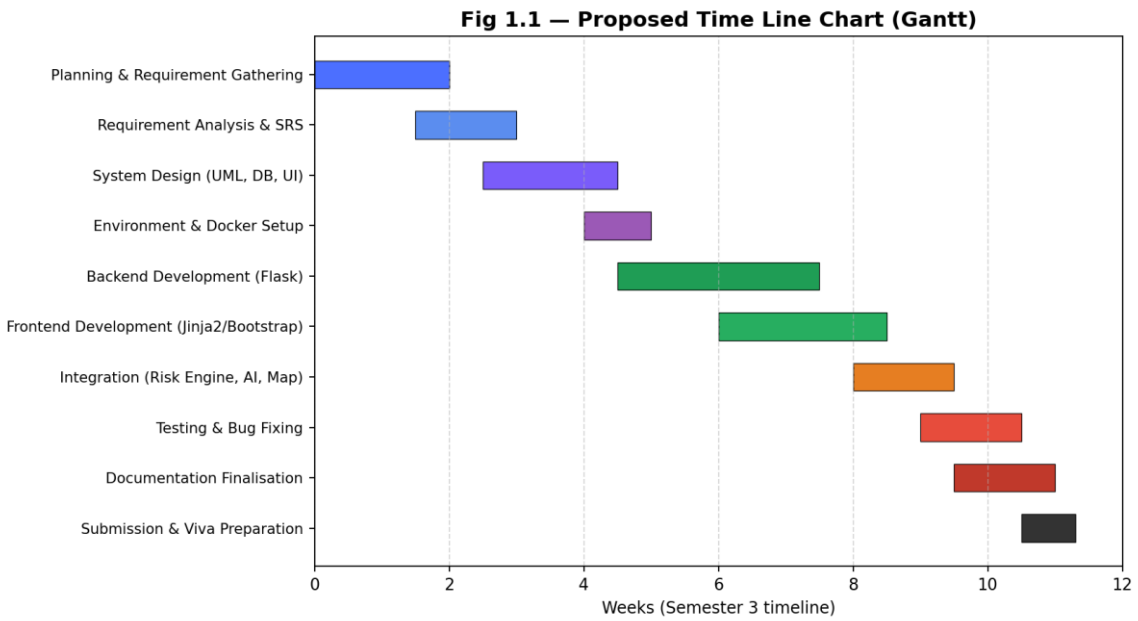


Fig 1.1 — Proposed Time Line Chart

1.9 Targeted User

- Logistics / Supply Chain Manager — the primary decision-maker who reviews and resolves disruptions.
- Field Operators — Captain, Pilot, Driver, and Loco Pilot — who execute the resolved route.
- System Administrator — manages users, hubs, cargo types, and system configuration.
- Indirectly, end customers and downstream partners who benefit from fewer missed deliveries.

2. REQUIREMENT DETERMINATION AND ANALYSIS

2.1 Requirement Determination

Requirements for ChainShield Transportation System were determined through a combination of group discussion, structured analysis of the Smart Supply Chains problem statement, and review of the team's earlier ChainShield prototype (built on FastAPI/React for a solution-building initiative) as a feature reference point.

The group broke the problem down into what a supply-chain manager needs to see, what a field operator needs to act on, and what an administrator needs to configure — and cross-checked each requirement against the objective of catching disruptions before they cascade, rather than after.

Key requirements identified through this process:

- All users (admin, manager, field operator) must authenticate before accessing the system.
- The system must continuously evaluate risk for every active shipment without manual triggering.
- A disruption must be created automatically once a shipment's risk score crosses a defined threshold.
- The system must propose at least two ranked route-solution alternatives per disruption.
- Field operators must be notified in-app the moment a manager resolves a disruption in their favour.
- Admins must be able to manage users, hubs, and cargo types, and view an audit trail of system activity.
- The system must be demonstrable without live production data, via a simulated/demo disruption trigger.

2.2 Requirement Specification

Functional Requirements:

- User registration, login, and role-based access control (Admin, Manager, Field Operator).
- Continuous, scheduled risk scoring of all active shipments.
- Automatic disruption detection and alerting once a risk threshold is crossed.
- Generation of ranked route-solution alternatives (reroute vs. hold) with cost, time, and risk trade-offs.
- In-app notification delivery to the assigned field operator upon disruption resolution.
- Direct messaging (chat) between users.
- Admin CRUD operations for users, logistics hubs, and cargo types.
- Activity audit log for all administrative and disruption-resolution actions.
- AI assistant for natural-language queries about the supply chain.
- Live map rendering of shipment positions and disruption zones.

- A one-click demo-mode trigger for presentation purposes.

Non-Functional Requirements:

- Performance — risk scores must refresh at least once every 60 seconds.
- Usability — each role must see a dashboard tailored to its responsibilities, without unnecessary clutter.
- Portability — the system must run identically on any machine via Docker, without manual environment setup.
- Reliability — database writes must be transactional; the background scheduler must recover gracefully from a failed run.
- Security — passwords must be hashed (never stored in plain text); sessions must be managed securely via Flask-Login.
- Maintainability — code must be organised into modular Flask blueprints so features can be extended independently.

Platform / Hardware & Software Requirements:

Operating System	Windows 11 / any OS supporting Docker
Backend Language	Python 3.10+
Web Framework	Flask, Flask-SQLAlchemy, Flask-Login, Flask-SocketIO, Flask-WTF
Database	SQLite
Containerisation	Docker, Docker Compose
Front-End Tools	HTML5, CSS3, Bootstrap 5, Jinja2, JavaScript, Leaflet.js, Chart.js
External APIs	Google Gemini API, OpenWeatherMap API
Documentation Tool	Microsoft Word 2021

3. SYSTEM DESIGN

3.1 Use Case Diagram

The diagram below summarises the primary use cases for each of the three system roles — Admin, Manager, and Field Operator — within the ChainShield Transportation System boundary.

Fig 3.2 — Use Case Diagram

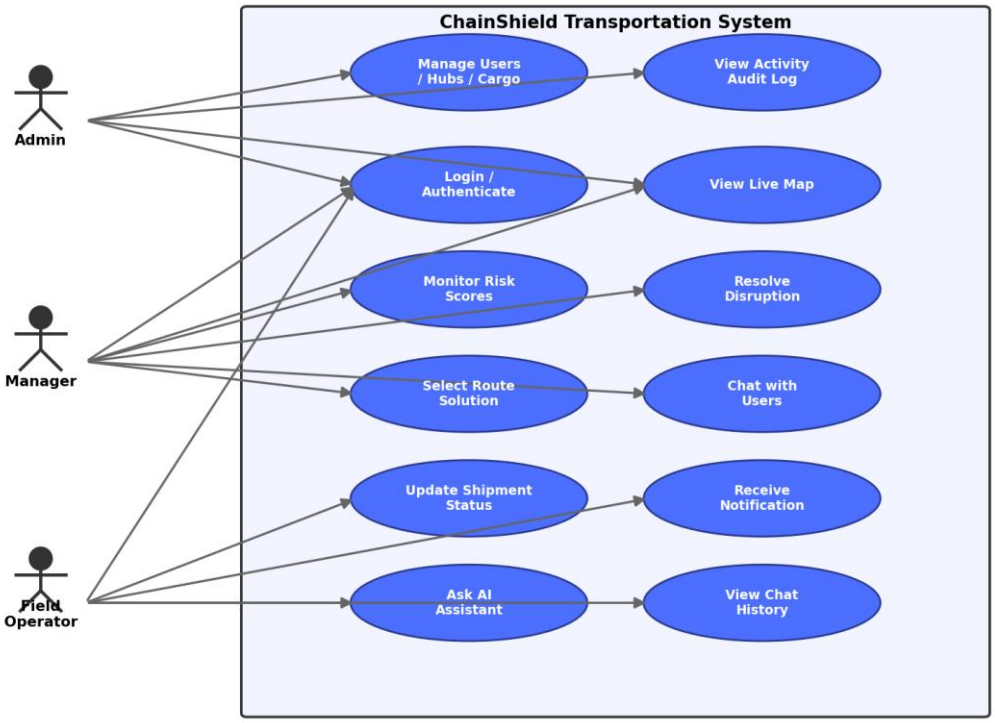


Fig 3.2 — Use Case Diagram

System Architecture:

The system follows a layered architecture: a Flask + Jinja2 + Bootstrap presentation layer on top, core services (Risk Engine, Route Solver, Notifications/Chat, AI Assistant, Admin Services) in the middle, a Flask-SQLAlchemy data access layer, and a SQLite database at the base — the entire stack packaged inside a Docker container for consistent deployment.

Fig 3.1 — ChainShield System Architecture

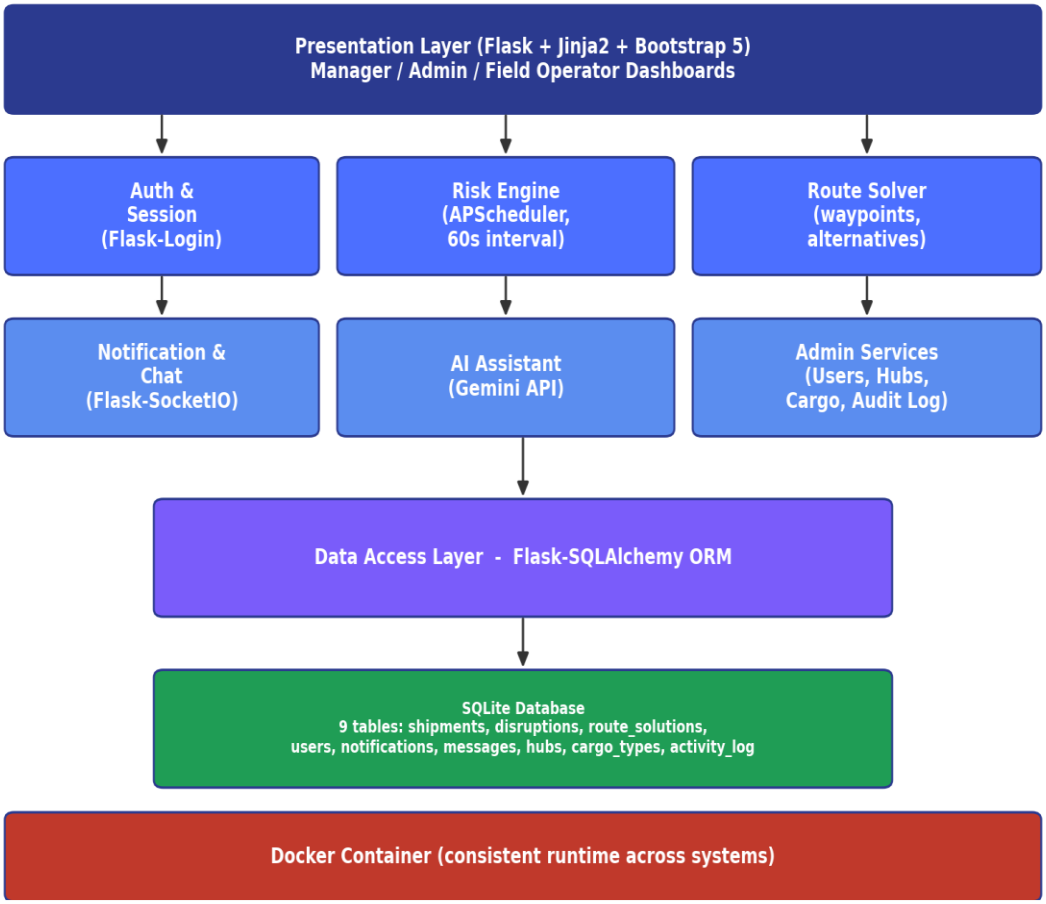


Fig 3.1 — ChainShield System Architecture

3.2 Data Dictionary

Database: chainshield

1. users

Column Name	Data Type	Description
id	INTEGER (PK)	Unique user identifier
name	VARCHAR(100)	Full name
email	VARCHAR(100)	Login email (unique)
password_hash	VARCHAR(255)	Hashed password (Werkzeug)
role	VARCHAR(20)	admin / manager / captain / pilot / driver / loco_pilot
phone	VARCHAR(20)	Contact number
created_at	DATETIME	Account creation timestamp

2. shipments

Column Name	Data Type	Description
id	INTEGER (PK)	Unique shipment identifier
tracking_no	VARCHAR(30)	Human-readable tracking number
origin / destination	VARCHAR(100)	Origin and destination hub names
cargo_type_id	INTEGER (FK)	References cargo_types.id
transport_mode	VARCHAR(20)	Road / Rail / Sea / Air
status	VARCHAR(20)	In-transit / At-risk / Disrupted / Delivered
current_lat / current_lng	FLOAT	Live simulated position
risk_score	FLOAT	Composite risk score (0-100)
risk_level	VARCHAR(15)	LOW / AT-RISK / MEDIUM / HIGH / CRITICAL
assigned_operator_id	INTEGER (FK)	References users.id
hub_id	INTEGER (FK)	References hubs.id
created_at	DATETIME	Shipment creation timestamp

3. disruptions

Column Name	Data Type	Description
id	INTEGER (PK)	Unique disruption identifier
shipment_id	INTEGER (FK)	References shipments.id
type	VARCHAR(30)	Weather / Congestion / Cargo / Historic
severity	VARCHAR(15)	AT-RISK / MEDIUM / HIGH / CRITICAL
risk_score	FLOAT	Score at time of detection
status	VARCHAR(15)	Open / Resolved
detected_at / resolved_at	DATETIME	Lifecycle timestamps
resolved_by	INTEGER (FK)	References users.id (manager)

4. route_solutions

Column Name	Data Type	Description
id	INTEGER (PK)	Unique solution identifier
disruption_id	INTEGER (FK)	References disruptions.id
option_no	INTEGER	1 or 2 (ranked alternative)
description	TEXT	e.g. “Divert via Antwerp”

extra_time_hours	FLOAT	Additional transit time
extra_cost	FLOAT	Additional cost (INR)
risk_score	FLOAT	Residual risk if chosen
selected	BOOLEAN	Whether the manager chose this option

5. notifications, messages, hubs, cargo_types, activity_log

Column Name	Data Type	Description
notifications.id / user_id / message / is_read / created_at	mixed	In-app alerts delivered to a specific user
messages.id / sender_id / receiver_id / chat_id / content / sent_at	mixed	Direct chat messages between users
hubs.id / name / location / lat / lng	mixed	Logistics hub master data
cargo_types.id / name / sensitivity_weight	mixed	Cargo master data used in risk scoring
activity_log.id / user_id / action / timestamp	mixed	Audit trail of administrative actions

3.3 User Interface Design

The interface is built with server-rendered Jinja2 templates and Bootstrap 5, sharing a common base layout across screens. Planned screens include:

- Login / Register — role-aware authentication screen.
- Manager Dashboard — live map, KPI summary cards, filterable shipment list, and an alerts panel.
- Disruption Resolution Modal — shows the ranked route solutions with time/cost/risk for each, and a one-click “Resolve” action.
- Field Operator Dashboard — the operator's assigned shipment, a status-update control, and their notification feed.
- Admin Panel — tabbed views for Users, Hubs, Cargo Types, and the Activity Audit Log.
- Chat Panel — a slide-out panel for direct messaging between users.
- AI Assistant Panel — a chat-style panel for natural-language queries about the supply chain.

All screens follow a consistent colour system (status colours for LOW/AT-RISK/MEDIUM/HIGH/CRITICAL) and are responsive across desktop and tablet breakpoints via Bootstrap's grid system.

3.4 Report Design

The system is designed to generate the following reports, viewable on-screen and exportable as CSV/PDF:

- Disruption Summary Report — count and type of disruptions over a selected date range.
- Shipment Risk Report — current risk score and level for every active shipment.
- Operator Response Time Report — time taken by each field operator to acknowledge and act on a resolved disruption.
- Activity Audit Log Export — a chronological record of all administrative and resolution actions, for accountability.

4. DEVELOPMENT

4.1 Coding Standards

The following coding standards will be followed once development begins, to keep the codebase consistent, readable, and easy for all three group members to work on in parallel:

- PEP 8 style guide for all Python code, with 4-space indentation.
- Modular Flask Blueprint structure — separate blueprints for auth, shipments, disruptions, admin, notifications, chat, and the AI assistant.
- Business logic kept out of route handlers — risk scoring, route solving, and weather integration live in a dedicated services/ package.
- Template inheritance via a single base.html in Jinja2, so shared layout (navbar, sidebar, footer) is defined once.
- Meaningful, descriptive names for variables, functions, and database columns — no single-letter names outside loop counters.
- Secrets and configuration (API keys, database URL) stored in environment variables via a .env file — never hard-coded.
- Docstrings and inline comments for non-trivial logic, particularly the risk-scoring formula and route-solution ranking.
- Git version control with feature branches and descriptive commit messages.
- Dependency versions pinned in requirements.txt for reproducible builds inside Docker.

5. TESTING

5.1 Test Cases

Testing will combine unit tests (individual functions such as the risk-scoring formula) with integration tests (full request/response cycles through Flask routes) and manual UI testing of each dashboard. Representative planned test cases are listed below; results will be filled in once development and testing are complete.

Test Case 1 — User Login

Test ID	TC001
Description	Verify that a registered user can log in with correct credentials.
Pre-conditions	User account exists in the database.
Test Data	email = manager@chainshield.com, password = correct password
Expected Result	User is authenticated and redirected to their role-specific dashboard.
Status	Pass / Fail (to be filled during testing)

Test Case 2 — Risk Score Calculation

Test ID	TC002
Description	Verify the risk engine correctly computes a composite score from the 4 weighted factors.
Pre-conditions	A shipment exists with known weather, congestion, cargo, and history inputs.
Test Data	Weather=80, Congestion=60, Cargo=50, History=40
Expected Result	Composite score = 0.40×80 + 0.25×60 + 0.20×50 + 0.15×40 = 63 (MEDIUM)
Status	Pass / Fail (to be filled during testing)

Test Case 3 — Automatic Disruption Creation

Test ID	TC003
Description	Verify a disruption record is auto-created once a shipment's risk score crosses the HIGH threshold (≥70).
Pre-conditions	Scheduler is running; a shipment's score is updated to 75.
Test Data	risk_score = 75
Expected Result	A new disruption row is created with 2 ranked route solutions; the manager is alerted.
Status	Pass / Fail (to be filled during testing)

Test Case 4 — Route Solution Selection & Notification

Test ID	TC004
Description	Verify that resolving a disruption notifies the assigned field operator instantly.
Pre-conditions	An open disruption with 2 route solutions exists; manager is logged in.
Test Data	Manager selects Option 1 (“Divert to Antwerp”)
Expected Result	Disruption status becomes Resolved; a notification appears in the field operator's dashboard.
Status	Pass / Fail (to be filled during testing)

Test Case 5 — Admin CRUD (Cargo Type)

Test ID	TC005
Description	Verify admin can add, edit, and delete a cargo type.
Pre-conditions	Admin is logged in.
Test Data	name = 'Pharma', sensitivity_weight = 0.9
Expected Result	Cargo type is saved and appears in the cargo type list; edits and deletion work as expected.
Status	Pass / Fail (to be filled during testing)

Test Case 6 — Docker Container Startup

Test ID	TC006
Description	Verify the application starts correctly inside a Docker container with no manual configuration.
Pre-conditions	Docker and Docker Compose are installed on the host machine.
Test Data	docker compose up
Expected Result	Flask app, SQLite database, and scheduler start successfully; the app is reachable on the configured port.
Status	Pass / Fail (to be filled during testing)

6. PROPOSED ENHANCEMENT

The following enhancements are identified as valuable future work, beyond the scope of the current MCA Semester 3 submission:

- Integration with real GPS/telematics hardware feeds in place of simulated shipment positions.
- Migration from SQLite to PostgreSQL to support multi-node, production-scale deployment.
- A predictive machine-learning model, trained on historical disruption data, to augment or replace the current weighted-heuristic risk formula.
- A companion mobile application for field operators, for on-the-go status updates and notifications.
- Multi-tenant support so multiple independent logistics companies can use isolated instances of the platform.
- Additional notification channels — SMS and WhatsApp — alongside in-app alerts.
- Integration with public customs and port-authority APIs for verified, real-time congestion data.

7. CONCLUSION

The ChainShield Transportation System addresses a well-defined, real-world problem: the tendency of supply-chain disruptions to be discovered only after delivery timelines are already compromised. By continuously scoring every active shipment against a transparent, weighted risk model, and by pairing every detected disruption with concrete, ranked route-solution alternatives, the system moves logistics management from a reactive posture to a preemptive one.

This project is being carried forward from an initial prototype built for a solution-building initiative, and rebuilt from scratch for MCA Semester 3 on a simpler, self-contained Python (Flask) and SQLite stack, containerised with Docker so that it runs identically on any system. The original, already-deployed prototype is left untouched; this rebuild lives in its own separate project folder.

Over the course of this semester, Group 59 aims to translate the requirement analysis and system design documented in this report into a working, testable application — demonstrating in practice how a modest, open-source technology stack can meaningfully address a genuinely difficult operational problem in global logistics.

8. BIBLIOGRAPHY

Books Referred

- Flask Web Development — Miguel Grinberg, O'Reilly Media
- Python Crash Course — Eric Matthes, No Starch Press
- Fundamentals of Database Systems — Ramez Elmasri & Shamkant B. Navathe, Pearson
- Software Engineering — Ian Sommerville, Pearson

Websites Referred

- https://flask.palletsprojects.com/
- https://docs.python.org/3/
- https://www.sqlite.org/docs.html
- https://docs.docker.com/
- https://getbootstrap.com/docs/5.3/
- https://leafletjs.com/reference.html
- https://developer.mozilla.org/
- https://ai.google.dev/gemini-api/docs
- https://openweathermap.org/api
- https://www.geeksforgeeks.org/